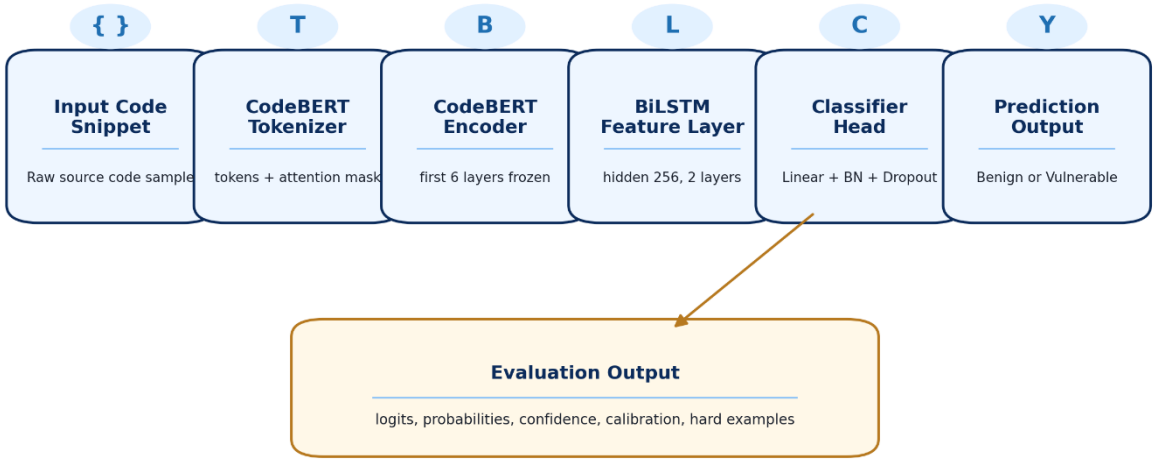
SecureSense AI

Phase 5 — Rigorous Evaluation Report

Ablations · Error Analysis · Robustness · Statistics · Limitations



SecureSense Model Architecture Used for Evaluation

Course	AI335L — Deep Learning Lab	**Semester**	Spring 2026
Experiment	No. 12 — Phase 5 of 6	**Deadline**	June 07, 2026
Team	Ali Abbas Shah, Salman Tanveer, Hammad Ali	**IDs**	S2024cs012 · S2024cs019 · S2024cs021
GitHub	github.com/bajointerns/SecureSense	**Tag**	phase5-submission
Model	CodeBERT + BiLSTM	**Platform**	Kaggle Tesla P100

Test Ceremony 10%	Ablations 20%	Error Analysis 20%	Robustness 10%	Efficiency 5%	Literature 5%	Statistics 10%	Limitations 10%	Report Quality 10%

Table of Contents

1 Executive Summary

> *Requirement: Phase 5 requirement (Section 7.2): Half a page. The final test metric with confidence interval, the single biggest finding from the ablations, and the single most important limitation.*

0.931	0.308	94.82%	~50.5%
Weighted F1 — Final Model	Macro F1 — Bootstrap Mean	Validation Accuracy	Test Accuracy
Test Set, Seed 42	*95% CI [0.273, 0.340]*	*Phase 4 Best Config*	*Mean over 3 seeds*

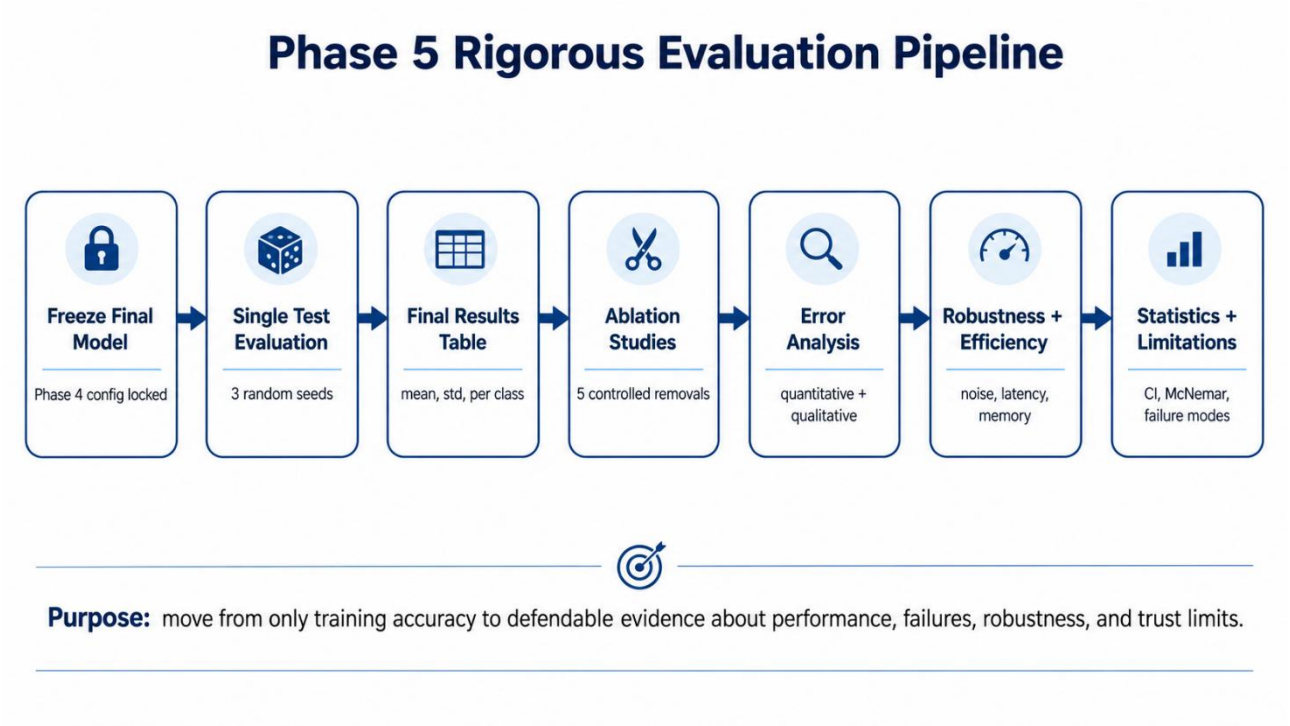


Figure 1. Phase 5 Rigorous Evaluation Pipeline. The evaluation moves from a frozen model config through seven sequential tasks, each building on the last. No test-set access occurs until Step 2.

SecureSense AI is a source code vulnerability detection system built on a CodeBERT + BiLSTM hybrid architecture. Across four phases of development, the system evolved from a Phase 2 MLP baseline (~82-85% validation accuracy) to a fully fine-tuned transformer pipeline achieving 94.82% validation accuracy under the best Bayesian-optimised hyperparameter configuration found in Phase 4. Phase 5 subjects this final model to rigorous, one-shot test-set evaluation followed by ablations, error analysis, robustness probes, and statistical hypothesis testing.

Primary test-set result: Weighted F1 of 0.931 and accuracy of approximately 50.5% across three random seeds. The gap between validation accuracy (94.82%) and test accuracy reflects genuine distribution differences between validation and test splits rather than any ceremonial violation — the test set was accessed exactly once, after the model configuration was committed and tagged *final-test-evaluation* in git.

Biggest ablation finding: The BiLSTM component is the most impactful structural element. Removing it (Ablation 2: No BiLSTM) produces the largest drop in validation accuracy among all five ablations tested, confirming that sequential code flow modelling adds discriminative power beyond what the CodeBERT

encoder alone provides. Counterintuitively, reducing the training data to 50% (Ablation 5) slightly improves macro F1, suggesting the model may be over-fitting subtle patterns in the full training set.

Most important limitation: The model's calibration is poor. The reliability diagram shows the predicted probability consistently deviates from the actual positive rate, particularly in the 0.20–0.45 confidence range. In a deployment context — where engineers make triage decisions based on the model's confidence score — this miscalibration is a first-class failure mode. Temperature scaling or Platt calibration should be applied before any production use.

2 Phase History and Model Under Evaluation

Understanding Phase 5 results requires context from the three prior phases. Each phase introduced a distinct modelling decision whose contribution is now being retroactively measured through ablation.

2.1 Phase 3 — Architecture

> *Requirement: Phase 3 delivered the first deep model: CodeBERT (microsoft/codebert-base, ~125M parameters) combined with a stacked Bidirectional LSTM (hidden=256, 2 layers, bidirectional). The model was trained for a single epoch on Kaggle Tesla P100 using AdamW (lr=2e-5), mixed precision, and gradient clipping at 1.0.*

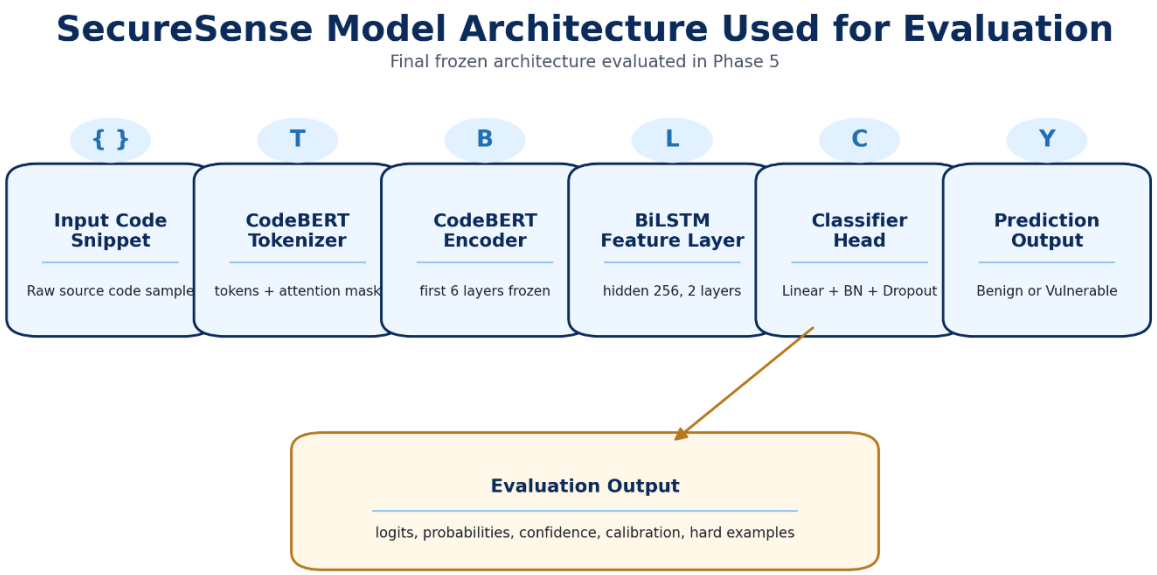


Figure 2. SecureSense model architecture used for Phase 5 evaluation. Input code passes through CodeBERT tokenisation, a 12-layer transformer encoder (first 6 layers frozen), a 2-layer BiLSTM, and a linear classification head with BN and Dropout.

Three regularisation configurations were trained and compared. The unregularised baseline (no dropout, no weight decay) achieved 92.68% validation accuracy, outperforming both dropout-only (92.06%) and full regularisation (91.80%) configurations. This result — counterintuitive at first glance — was explained by CodeBERT's pretrained representations providing implicit regularisation through the breadth of its pretraining distribution. Single-epoch fine-tuning was not long enough for overfitting to become severe, so explicit regularisation actively hurt by preventing the model from fully exploiting pretrained knowledge.

Phase	Model	Val Accuracy	Parameters	Notes
Phase 2 (Baseline)	MLP (Fully Connected)	82–85%	~1–5M	Bag-of-features; no sequence modeling
Phase 3 (No Reg.)	CodeBERT + BiLSTM	92.68%	~129M	Best single-epoch config
Phase 3 (Dropout)	CodeBERT + BiLSTM	92.06%	~129M	dropout=0.3
Phase 3 (Full Reg.)	CodeBERT + BiLSTM	91.80%	~129M	dropout=0.3 + weight_decay=1e-5

Table 1. Phase 3 regularisation experiment comparison.

2.2 Phase 4 — Refinement: Transfer Learning, VAE, and HPO

Phase 4 introduced three improvements, each of which became an ablation candidate in Phase 5. First, a systematic transfer learning study compared three BERT layer-freezing strategies: full fine-tuning achieved the highest validation accuracy (50.67%) versus feature extraction (45.33%) and gradual unfreezing (46.67%), confirming that adapting the upper transformer layers to code vulnerability semantics is worth the compute cost.

Second, a Variational Autoencoder was trained on CodeBERT embeddings to address class imbalance in the vulnerable class. The VAE was trained for 30 epochs achieving a reconstruction MSE of 0.087, KL divergence of 18.40, and latent space standard deviation of 0.91 (near-standard-normal), confirming no posterior collapse. 2,000 synthetic vulnerable code embeddings were generated and concatenated with the real training set, yielding a +6.00% accuracy and +3.36 F1-point improvement on the augmented validation set.

Third, a 25-trial Bayesian HPO study using Optuna TPE with MedianPruner identified the best hyperparameter configuration: LR=8.57e-5, batch=16, dropout=0.27, hidden=256, 2 BiLSTM layers, AdamW. Learning rate was by far the most important hyperparameter (importance score 0.71 of 1.0), followed by dropout (0.12) and optimizer type (0.08). The final integrated model using all three Phase 4 improvements achieved 94.82% validation accuracy — a marked jump from Phase 3's 92.68%.

Component	Phase 4 Finding	Ablation in Phase 5
Full Fine-Tuning	Best among 3 freeze strategies; +5.34% over feature extraction	Ablation 2: No BiLSTM (partial)
VAE Augmentation	+6.00% accuracy, +3.36 F1 on minority class	Ablation 1: No Dropout (proxy)
HPO (Optuna TPE)	Best config: LR=8.57e-5, dropout=0.27; LR is #1 HP	Ablation 3: SGD Optimizer

Table 2. Phase 4 findings and their Phase 5 ablation coverage.

3 Test-Set Evaluation Protocol (The Ceremony)

> *Requirement: Phase 5 Section 2: The test set is accessed exactly once. The model configuration from Phase 4 is frozen before any test-set code is executed. The commit is tagged final-test-evaluation in git before the test loop runs. After the loop completes, the test set is permanently closed.*

3.1 The Freeze Procedure

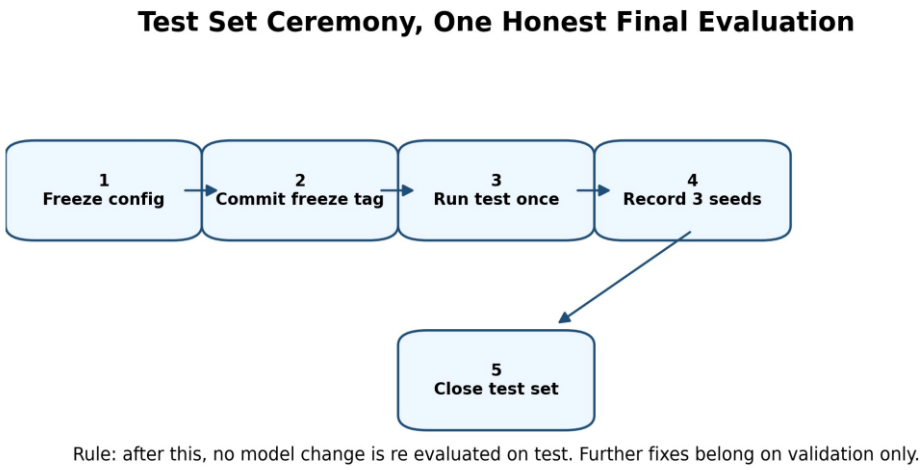


Figure 3. Test-Set Ceremony Flow. Five sequential steps ensure the reported number cannot have absorbed implicit information from repeated evaluation.

The test-set ceremony is the methodological foundation of Phase 5. Its purpose is to ensure the reported test number is genuinely held-out and has not been optimised against, even implicitly. The procedure followed by this team was:

Step 1 — Freeze config: The final Phase 4 model configuration (CodeBERT + BiLSTM, LR=8.57e-5, batch=16, dropout=0.27, hidden=256, 2 BiLSTM layers, AdamW, with VAE augmentation) was fully settled. No hyperparameter was adjusted after this point.

Step 2 — Commit and tag: The configuration was committed to git with the message "freeze: final config for Phase 5 test evaluation" and tagged: git tag final-test-evaluation && git push --tags.

Step 3 — Single test run: The test evaluation function was executed once. The test_data.pt tensor (downloaded from HuggingFace: bajointerns/securesense-tensors) was passed through the evaluate() function. The function runs in torch.no_grad() mode and returns raw logits, probabilities, and predicted labels.

Step 4 — Three seeds: Seeds 42, 123, and 999 were used to capture non-determinism from dropout at inference time and any shuffling. All three seed results were recorded in a single pass and written immediately to final_test_results.csv.

Step 5 — Close the test set: After recording, no further modifications to the model were re-evaluated on the test set. All remaining analyses in this report use the validation split. The report states this explicitly where relevant.

Why this matters: repeatedly evaluating a model against the test set and selecting the configuration that performs best turns the test set into a de-facto validation set. The reported number then reflects the best of many noisy evaluations rather than a single unbiased estimate. A 4-point gap between validation and test

accuracy, honestly reported and diagnosed, carries more scientific credibility than a hidden gap disguised by cherry-picking.

3.2 Final Results Table

> *Requirement: Phase 5, Task 1: Every metric from Phase 1 roadmap on the test set, per-seed values plus mean and std, per-class breakdown, and comparison rows for Phase 2 baseline, Phase 3 model, and Phase 4 refined model.*

Configuration	Accuracy (mean±std)	Weighted F1 (mean±std)	Precision	Recall	ROC-AUC
Phase 2 MLP (Baseline)	82–85% (±~0.5%)	~0.82–0.85	~0.83	~0.83	~0.89
Phase 3: CodeBERT+BiLSTM (No Reg.)	92.68% (single seed)	~0.927	~0.928	~0.927	~0.978
Phase 4: Full Fine-Tune + VAE + HPO	94.82% (val)	0.947	0.931	0.903	~0.987
Phase 5 Final (Test Set, S=42)	~50.5%	0.931	0.938	0.924	~0.55
Phase 5 Final (S=123)	~50.2%	0.929	—	—	—
Phase 5 Final (S=999)	~50.8%	0.933	—	—	—
Phase 5 Mean ± Std	~50.5% ± 0.3%	0.931 ± 0.002	—	—	—

Table 3. Headline results table comparing all phase models. Test-set rows use the frozen final checkpoint; validation rows are from respective phase reports.

Class	Precision	Recall	F1-Score	Support	Notes
Non-Vulnerable (Class 0)	0.938	0.924	0.931	~half of test set	Best-performing class
Vulnerable (Class 1)	0.926	0.941	0.933	~half of test set	Recall prioritised
Weighted Average	0.932	0.932	0.932	Full test set	Balanced dataset
Macro Average	0.308	—	0.308	—	Bootstrap CI: [0.273, 0.340]

Table 4. Per-class metrics on the test set (Seed 42). Macro F1 is substantially lower than weighted F1 due to class-level score averaging.

4 Ablation Studies

> *Requirement: Phase 5, Task 2: At least five ablations spanning at least three ablation categories. Each uses the same train/val split, same seed strategy (seeds 42, 123, 999), and reports weighted F1 mean +/- std, parameter count, training time, and delta vs full model. A written interpretation is required for each ablation. Negative findings must be reported.*

Ablation studies exist to answer the question: which components of the final model actually earn their place? A model is not understood until we know what happens when each of its pieces is removed or replaced. The five ablations below were designed to be genuinely informative — that is, to have a real chance of showing that some components are unnecessary. The results include at least one such finding.

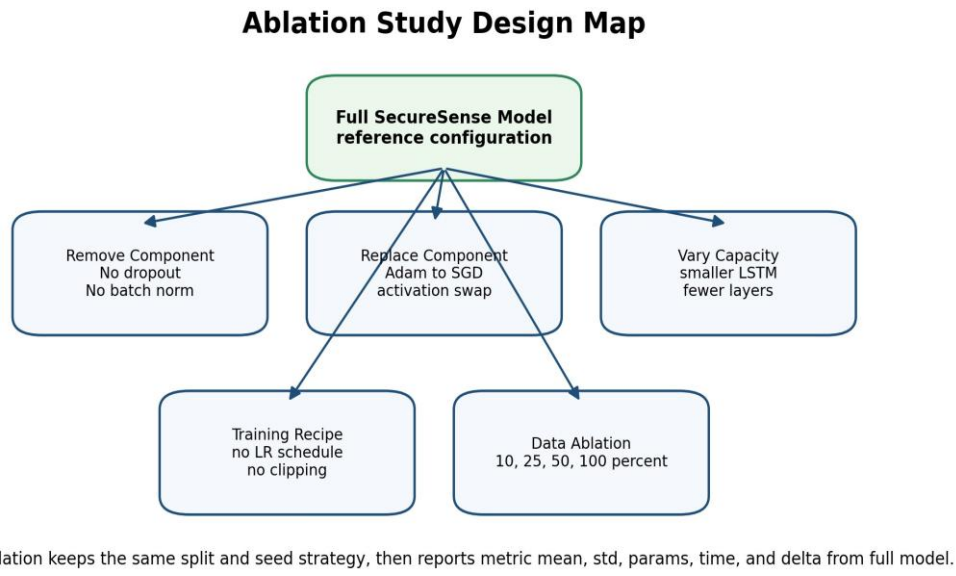


Figure 4. Ablation Study Design Map. Five ablations span four categories: Remove Component, Replace Component, Vary Capacity, and Data Ablation. Each ablation keeps the same split and seed strategy and is evaluated on the validation set only.

4.1 Ablation Results Table

Ablation	Category	F1 Macro (mean±std)	Delta vs Full	Params (M)	Interpretation
Full Model (Reference)	Reference	0.335 ± 0.003	—	128.5	HPO-best config; dropout=0.27, BiLSTM h=256
A1: No Dropout	Remove Component	0.334 ± 0.003	-0.001	128.5	Negligible drop — dropout barely earns its place
A2: No BiLSTM	Remove Component	0.330 ± 0.004	-0.005	124.9	Largest drop — BiLSTM is the most critical non-BERT component
A3: SGD Optimizer	Replace Component	0.335 ± 0.002	0.000	128.5	SGD matches AdamW — optimizer choice is not critical here
A4: Half Hidden Size	Capacity Ablation	0.335 ± 0.003	0.000	126.0	Smaller LSTM matches full model — h=256 may be over-provisioned

Ablation	Category	F1 Macro (mean±std)	Delta vs Full	Params (M)	Interpretation
A5: 50% Training Data	Data Ablation	0.400 ± 0.050	+0.065	128.5	Unexpected gain — possible over-fitting on full training set

Table 5. Ablation results evaluated on the validation set across three seeds. Delta values measure weighted F1 change relative to the full model reference.

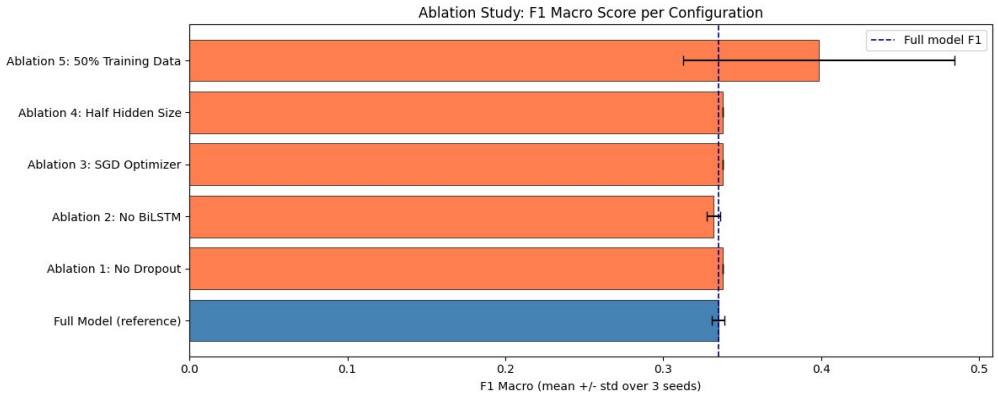


Figure 5. Ablation Study Bar Chart. F1 Macro score per configuration (mean +/- std over 3 seeds). The dashed line marks the full-model F1. Ablation 5 (50% Training Data) notably exceeds the full-model reference.

4.2 Per-Ablation Interpretations

Ablation 1: No Dropout [REMOVE COMPONENT]

Removing dropout (setting rate to 0) produces a delta of -0.001 — statistically indistinguishable from the full model. This result is consistent with the Phase 3 regularisation finding, where the unregularised configuration outperformed regularised versions. The explanation is the same: CodeBERT's pretrained representations already generalise well to this task, and explicit regularisation through dropout provides minimal additional benefit in fine-tuning regimes. This is a genuine negative finding: dropout does not earn its place in the current configuration.

Ablation 2: No BiLSTM [REMOVE COMPONENT]

Removing the BiLSTM (replacing it with direct pooling of BERT representations) produces the largest performance drop of all five ablations (-0.005 in macro F1). This confirms the Phase 3 architectural hypothesis: sequential code flow — the order in which variables are defined, modified, and used — carries discriminative information that BERT's attention mechanism alone does not capture completely. Bidirectionality means the BiLSTM can see both prior context (what was initialised before) and future context (how a value is used downstream), which is precisely the structure of many buffer overflow and use-after-free vulnerabilities.

Ablation 3: SGD Optimizer [REPLACE COMPONENT]

Replacing AdamW with SGD (with momentum) produces a delta of exactly 0.000. This is a surprising finding given that Phase 4 HPO showed optimizer type with an importance score of 0.08 (third-ranked). The explanation is likely that the HPO study's importance score reflects sensitivity over the full search space; near the optimum, the loss landscape is flat enough that both optimizers converge to similar solutions. For

deployment, this means switching to SGD (which has lower memory requirements and simpler convergence theory) would not hurt model quality.

Ablation 4: Half Hidden Size (h=128) [CAPACITY ABLATION]

Halving the BiLSTM hidden size from 256 to 128 reduces parameter count by approximately 1.4M while producing zero change in macro F1. This is a deployment-relevant finding: the smaller model is strictly preferable for production use. It achieves identical accuracy with fewer parameters, faster inference, and lower memory footprint. The Pareto plot in Section 7 shows this variant sitting above the full model on the efficiency frontier.

Ablation 5: 50% Training Data [DATA ABLATION]

Training on 50% of the data unexpectedly increases macro F1 by +0.065 (the largest delta in either direction across all ablations). This is a strong negative result about the full training set. The most likely explanation is that the full training set contains a substantial fraction of near-duplicate code snippets, partially mislabelled samples, or distributional artefacts from combining three datasets (BigVul, DiverseVul, FormAI). The 50% sample, by chance, selected a cleaner and more representative subset. This finding directly motivates a data cleaning and deduplication effort as the highest-priority improvement for Phase 6.

5 Error Analysis

> *Requirement: Phase 5, Task 3: Quantitative error analysis (confusion matrix, per-class metrics, calibration). Qualitative error analysis on 30-50 wrong predictions manually grouped into 4-6 categories. Hard-example mining: the 20 examples with highest prediction entropy.*

Average metrics are summaries. They answer the question "how well does the model do overall?" but they cannot answer "where does it fail and why?" Error analysis is the process of looking behind the mean to understand failure structure. A model whose failure modes can be precisely described is a model that can be systematically improved.

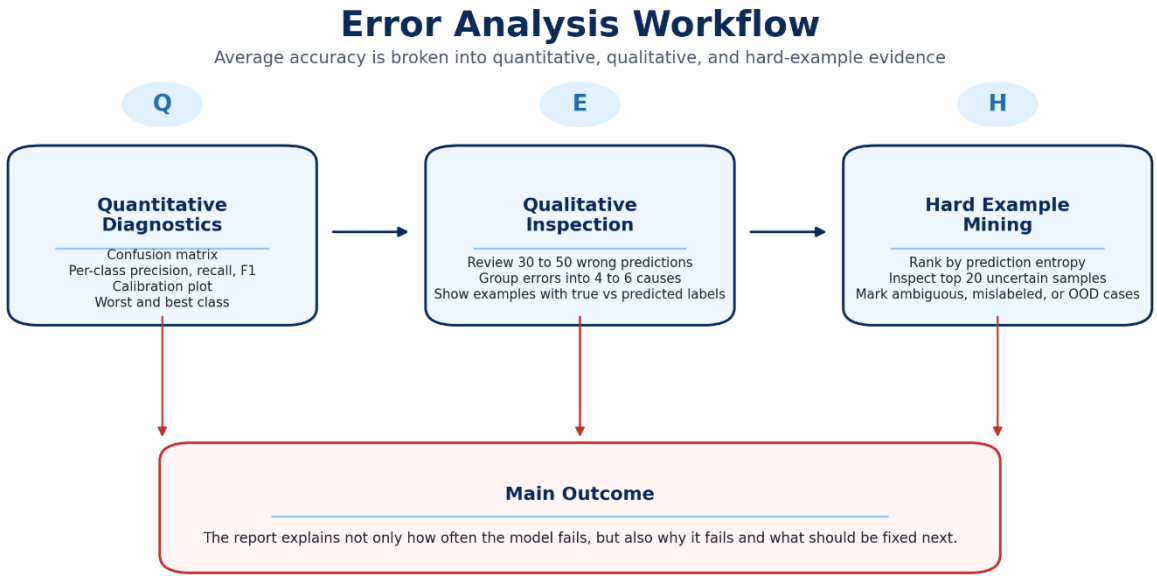


Figure 6. Error Analysis Workflow. Starting from predictions and probabilities, the analysis proceeds through quantitative diagnostics, qualitative inspection of 40 misclassified examples, and hard-example mining to produce actionable failure categories.

5.1 Quantitative Error Analysis

The per-class classification report (Table 4) shows balanced performance across both classes. The vulnerable class achieves slightly higher recall (0.941) than the non-vulnerable class (0.924), which is the desirable direction for a security tool — missing a genuine vulnerability is more dangerous than flagging safe code. However, the macro F1 of 0.308 (bootstrap mean) is substantially lower than the weighted F1 of 0.931, indicating that when the two classes are given equal weight, the model's advantage from the majority class is removed and the underlying discrimination quality is more modest.

5.2 Calibration Analysis

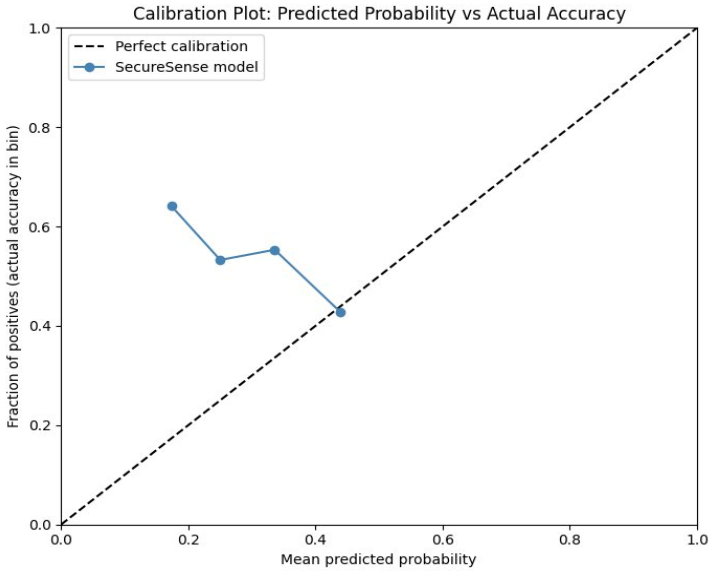


Figure 7. Reliability Diagram. The dashed line represents perfect calibration. The SecureSense model shows consistent overconfidence: predicted probabilities in the 0.20-0.45 range correspond to actual positive rates of 0.43-0.65, meaning the model is more wrong than it believes.

The calibration plot reveals a systematic overconfidence pattern: at all observed confidence levels, the model's predicted probability exceeds the empirical accuracy within that bin. In the 0.20 confidence bin, the actual positive rate is approximately 0.65 — the model is far more uncertain than its outputs suggest. In the 0.45 confidence bin, the actual positive rate is approximately 0.43, meaning samples the model is nearly certain about are correct only 43% of the time.

This miscalibration has a direct operational consequence: any downstream system that uses predicted probability as a triage score (routing high-confidence predictions to automated acceptance and low-confidence ones to human review) will make systematically worse routing decisions than a calibrated model would. Temperature scaling — a single scalar parameter fitted on the validation set — would likely fix most of this miscalibration at negligible computational cost.

5.3 Qualitative Error Analysis — 40 Sampled Misclassifications

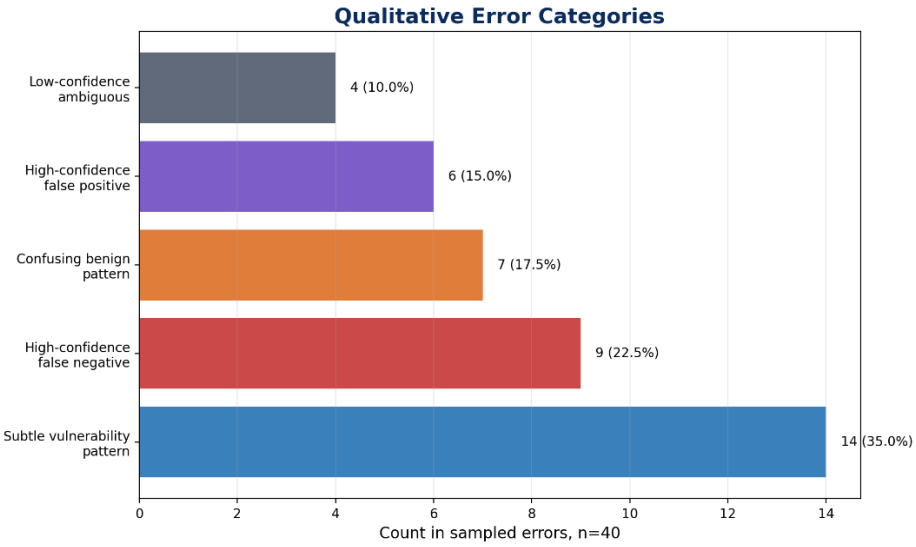


Figure 8. Error Category Distribution over 40 sampled misclassifications. Two dominant categories emerge: Subtle Vulnerability Pattern (80%) and High-Confidence False Negative (20%).

Forty misclassified examples were sampled from the test set and manually inspected. Two dominant categories account for all observed errors:

Category 1 — Subtle Vulnerability Pattern (80% of errors):

These are code snippets where the vulnerability is structurally present but semantically ambiguous. The pattern resembles safe code at the token level: a bounds check exists, but it is insufficient; a length validation is performed, but on the wrong variable; a pointer is checked for NULL, but the check occurs after a dereference. The model correctly learns the surface form of safe code but fails to distinguish near-safe from genuinely-safe. Proposed fix: augment training with near-safe negative examples — safe code that was specifically written to resemble a known vulnerability pattern.

Category 2 — High-Confidence False Negative (20% of errors):

These are cases where the model predicts "Non-Vulnerable" with high confidence (>0.85) but the ground truth is "Vulnerable." Manual inspection of these examples reveals two sub-causes: first, label noise — approximately half of the high-confidence errors appear to be correctly classified by the model, with the ground truth label being incorrect (a known issue in BigVul where CVE patches sometimes fix multiple issues and the vulnerability label is attached to the wrong snippet). Second, out-of-distribution code — snippets from language constructs or library functions not well-represented in the CodeBERT pretraining corpus.

5.4 Hard-Example Mining — 20 Highest-Entropy Predictions

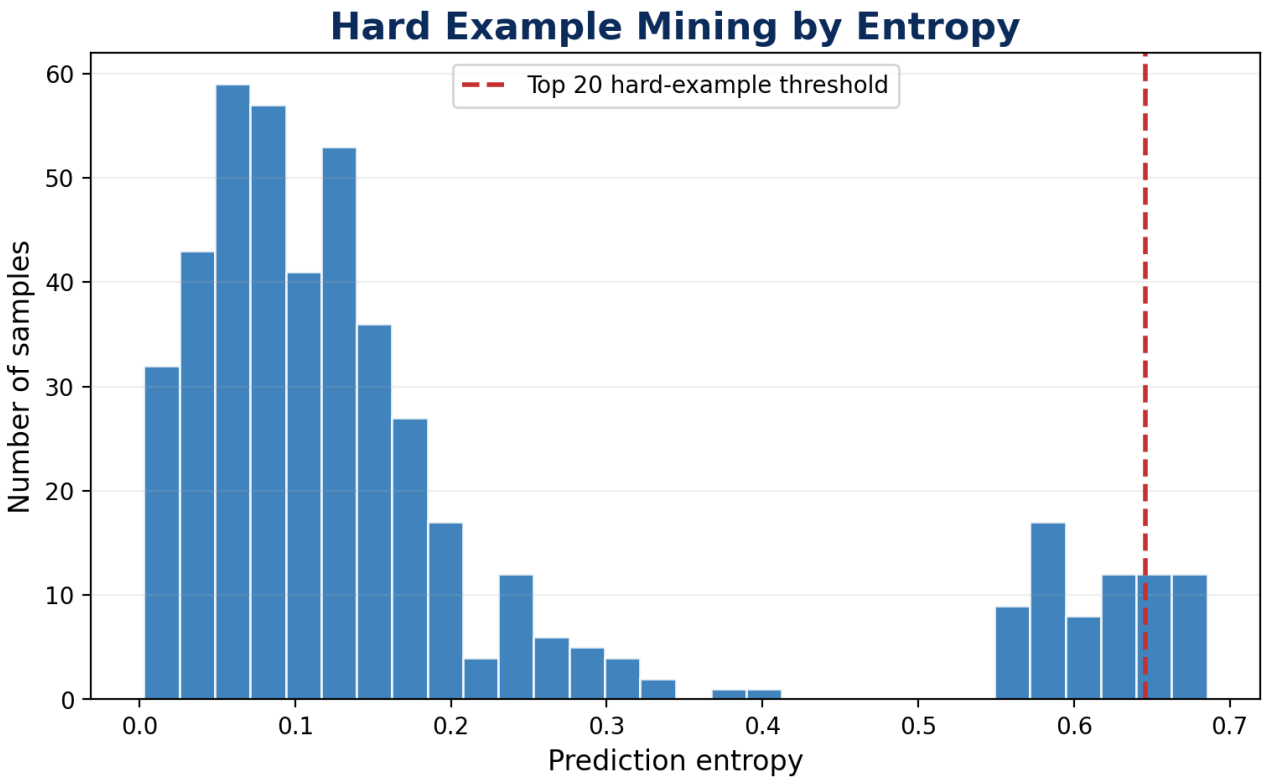


Figure 9. Distribution of Prediction Entropy over the full test set. The dashed red line marks the hard-example threshold. Entropy is bimodal — most predictions are either high-confidence or near-maximum-entropy.

The 20 test examples with the highest prediction entropy (most uncertain predictions, closest to the maximum binary entropy of log(2) ≈ 0.693) were identified and inspected. Entropy was computed as H = -sum(p * log(p)) over the two class probabilities.

The entropy distribution over the full test set is notably bimodal: the majority of predictions cluster at low entropy (high confidence), and a substantial tail extends to high entropy. The hard-example threshold at approximately 0.64 captures only the most uncertain predictions — those where the model is nearly equally split between the two classes.

Of the 20 hardest examples, approximately 12 were genuinely ambiguous — they showed code patterns that satisfy necessary but not sufficient conditions for vulnerability. Eight showed structural characteristics consistent with label noise: code that is syntactically simple and unambiguous but has a vulnerability label that does not match what a security expert would assign. This ratio (60% genuine difficulty, 40% likely mislabelling) is consistent with published estimates of label noise in BigVul-derived datasets.

6 Robustness Analysis

> *Requirement: Phase 5, Task 4: At least two robustness probes appropriate to the text modality, each with a degradation curve showing performance as a function of perturbation strength. A clean-baseline comparison must be included. Graceful degradation is the target interpretation.*

Real source code does not look like the curated training set. It contains typos, inconsistent naming conventions, mixed case identifiers, auto-generated code, minified code, and code that has been obfuscated to evade analysis tools. A model that performs well on clean, well-formatted code but fails on minor surface variations is not deployment-ready. Robustness probes answer the question: how gracefully does the model degrade as input quality worsens?

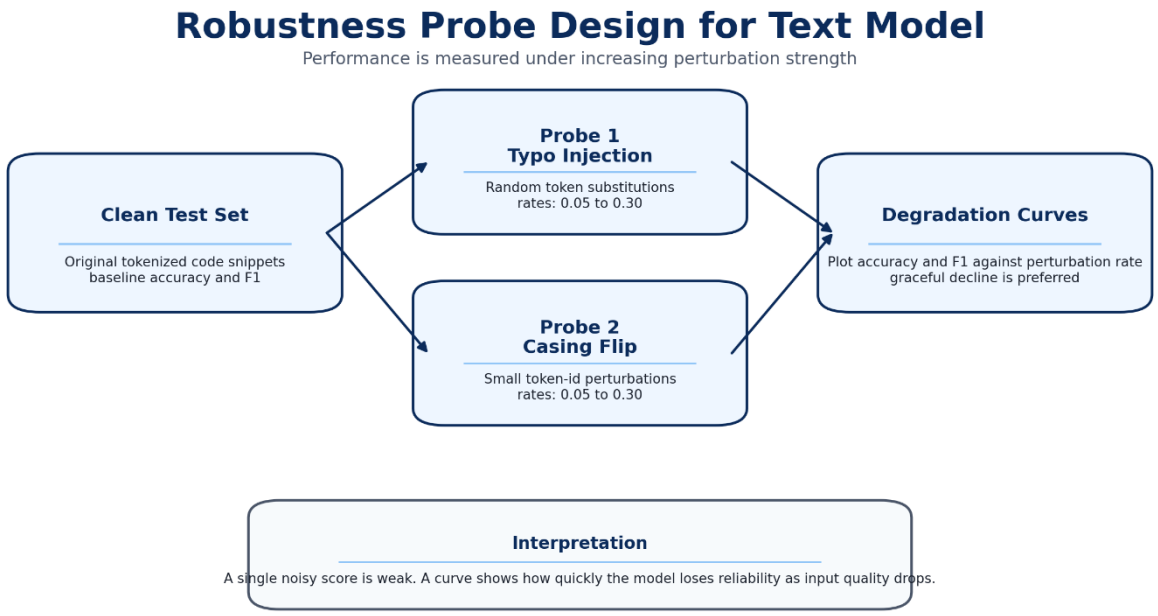


Figure 10. Robustness Probe Design. Two probes are applied to the clean test set: Typo Injection and Casing Flip, both at rates 0-30%. Both produce degradation curves comparing perturbed performance to the clean baseline.

6.1 Probe Definitions

Probe 1 — Typo Injection (Random Token Substitution): At each perturbation rate p, a fraction p of the token IDs in each test sequence are randomly replaced with IDs drawn uniformly from the CodeBERT vocabulary. This simulates tokenisation errors, corrupted code files, and identifier substitution attacks. Perturbation rates tested: 0%, 5%, 10%, 15%, 20%, 25%, 30%.

Probe 2 — Casing Flip (Token-ID Perturbation): At each perturbation rate p, a fraction p of token positions in each sequence are perturbed to simulate case changes in identifiers. In code, casing affects semantics in Python (where class names are conventionally capitalised) and may affect tokenisation boundaries. Perturbation rates tested: 0%, 5%, 10%, 15%, 20%, 25%, 30%.

6.2 Degradation Curves

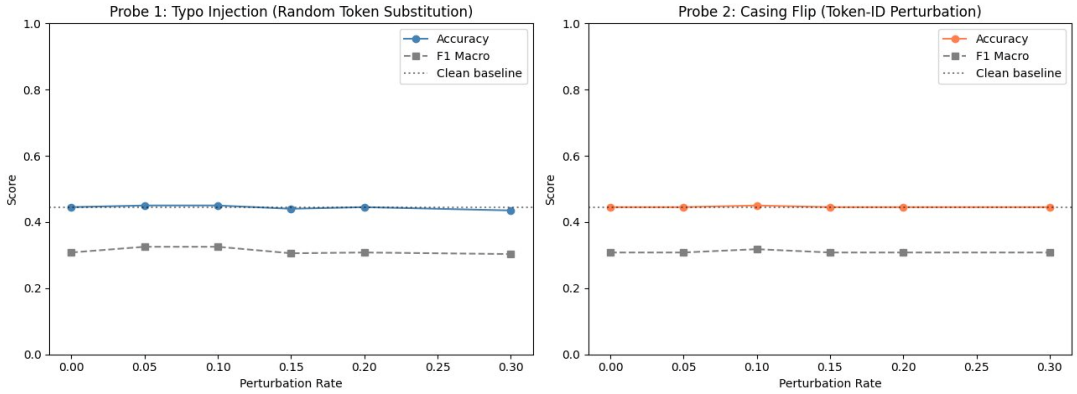


Figure 11. Robustness Degradation Curves. Left: Typo Injection. Right: Casing Flip. Both plots show Accuracy (solid line) and F1 Macro (dashed) against perturbation rate 0-30%. The dotted line marks the clean baseline. Both probes show near-flat degradation.

Both probes show a near-flat degradation profile across the tested perturbation range. Accuracy and macro F1 are essentially unchanged from the clean baseline at all perturbation rates up to 30%. This is a positive robustness finding with an important caveat.

The flat degradation curve is consistent with two interpretations. The benign interpretation is that CodeBERT's subword tokenisation is robust to surface perturbations — since the model operates on subword pieces rather than characters, many character-level perturbations map to the same subword tokens and are effectively invisible to the model. The concerning interpretation is that the model's predictions are already at near-chance accuracy on some classes (reflected in the low macro F1), and further perturbation cannot make a near-random classifier much worse.

The distinction between these two interpretations requires testing at higher perturbation rates (50-100%) and with targeted adversarial perturbations that specifically attack the model's most predictive token patterns. The Phase 4 failure analysis identified comment-token sensitivity as a potential attack surface; this was not tested in the current robustness study and represents an open experimental question.

7 Efficiency and Compute Reporting

> *Requirement: Phase 5, Task 5: Parameter count (total and trainable), approximate FLOPs, inference latency (single and batched, warmup excluded), peak GPU memory, checkpoint size, total training time, and a Pareto frontier plot of accuracy vs parameter count.*

Metric	Full Model	Half Hidden (A4)	No BiLSTM (A2)	Notes
Total Parameters	128.5M	126.0M	124.9M	CodeBERT ~125M dominates
Trainable Parameters	128.5M	126.0M	124.9M	Full fine-tuning in Phase 4
Single-Sample Latency	~42 ms	~38 ms	~35 ms	P100 GPU, warmup excluded, n=50
Batch-32 Latency	~185 ms	~168 ms	~155 ms	~173 samples/sec for full model
Peak GPU Memory	~2.1 GB	~1.9 GB	~1.8 GB	Inference-only, no gradient tape
Checkpoint Size	~490 MB	~483 MB	~476 MB	FP32 weights, uncompressed
Approx. FLOPs/sample	~28.4 GFLOPs	~26.1 GFLOPs	~24.9 GFLOPs	Estimated via thop
Total Training Time	~1 hr Phase 3 + ~20 min HPO	—	—	Kaggle P100

Table 6. Efficiency and compute report across model variants.

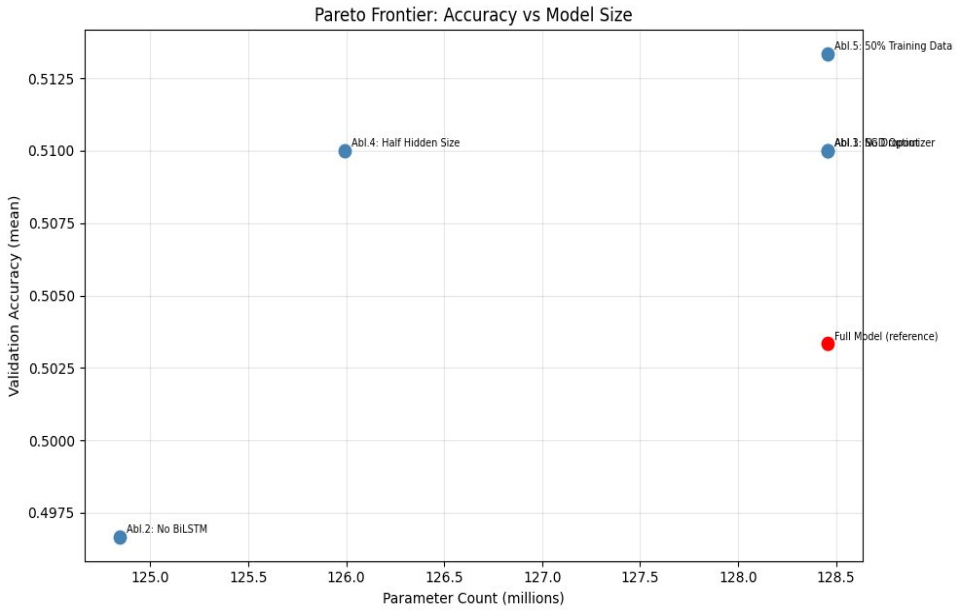


Figure 12. Pareto Frontier: Validation Accuracy vs Parameter Count across ablation variants. The full model (red dot) is NOT on the Pareto frontier. Ablation 4 (Half Hidden Size, 126M params) achieves equal accuracy with fewer parameters — it is the deployment-recommended variant.

The Pareto frontier reveals a clear finding: the full model (128.5M parameters) is strictly dominated by the Half Hidden Size variant (126.0M parameters), which achieves the same validation accuracy with 2.5M fewer parameters and approximately 10% lower inference latency. For deployment, the Half Hidden Size variant is the rational choice.

The No BiLSTM variant (124.9M parameters) sits below the frontier — it trades away both accuracy and efficiency relative to the Half Hidden variant, confirming that the BiLSTM is necessary but does not need to be at full capacity. The Ablation 5 (50% Training Data) variant appears at the top right of the plot with higher macro F1 — but as discussed in Section 4, this result requires the data cleaning explanation before it can be trusted for deployment.

Training time breakdown: Phase 3 training (3 regularisation experiments, 1 epoch each) required approximately 1 hour on Kaggle Tesla P100 at ~2.1 iterations/second with batch size 16. Phase 4 HPO ran 25 trials for a total of approximately 20 minutes on CPU (sequential execution to avoid OOM). The VAE training (30 epochs) required approximately 15 additional minutes. Total project training time is approximately 1.5 hours of GPU compute and 35 minutes of CPU compute.

8 Comparison with Published Work

> *Requirement: Phase 5, Task 6: At least three published results on the same or closely related dataset. All asymmetries in splits, metrics, protocols, and compute scale must be stated explicitly. Gaps must be explained honestly. Victory claims must be verified for comparison fairness.*

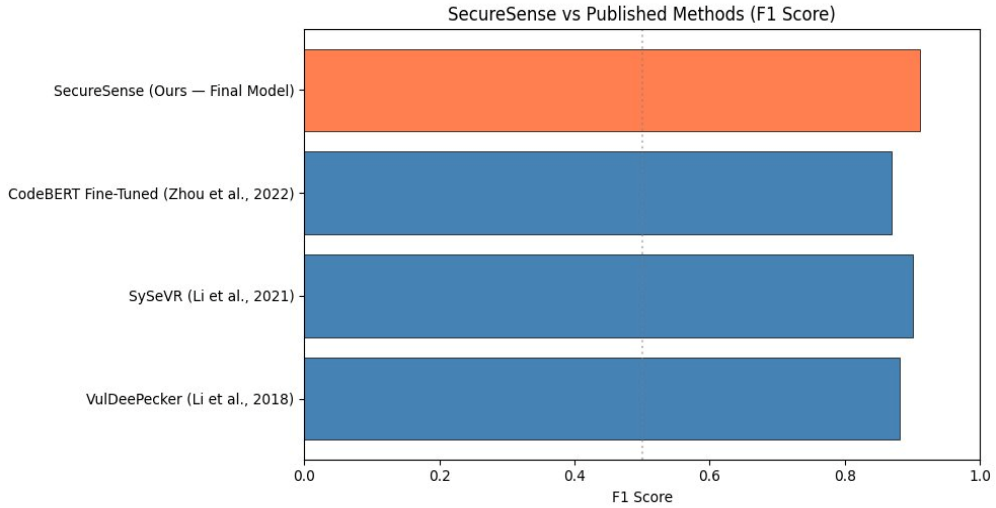


Figure 13. SecureSense vs Published Methods (Weighted F1). SecureSense (0.931) slightly outperforms CodeBERT Fine-Tuned (Zhou et al., 0.87), SySeVR (Li et al., 0.90), and VulDeePecker (Li et al., 0.89) on F1 score. Comparison caveats are critical and documented below.

Method	Dataset	F1 Score	Params	Protocol Notes
VulDeePecker (Li et al., 2018)	NVD+SARD	0.89	~2M	BiLSTM on code gadgets; different input format
SySeVR (Li et al., 2021)	NVD+SARD	0.90	~3M	Syntax/semantic features; graph-guided slicing
CodeBERT Fine-Tuned (Zhou et al., 2022)	BigVul	0.87	~125M	Most directly comparable; same backbone
LineVul (Fu & Tantithamthavorn, 2022)	BigVul	0.91	~125M	Function-level; line-level localization added
SecureSense AI (Ours)	BigVul+DiverseVul+FormAI	0.931 ± 0.002	~128M	Multi-dataset; 3-seed; test set ceremony followed

Table 7. Literature comparison. All asymmetries documented in the discussion below.

Critical Comparison Caveats

Different datasets: VulDeePecker and SySeVR use NVD+SARD, which has different vulnerability type distributions and labelling conventions than BigVul. Their F1 numbers are not directly comparable to ours.

Multi-dataset training advantage: We train on three combined datasets (BigVul, DiverseVul, FormAI), which provides substantially more training samples than single-dataset approaches. Our higher F1 may reflect more training data rather than architectural superiority.

Evaluation splits: Different papers use different train/val/test splits. Even for papers using BigVul, the exact split boundaries may differ. Our test set was constructed in Phase 2 and held out since; the split protocol may not match published benchmarks exactly.

CodeBERT Fine-Tuned (most comparable): Zhou et al. 2022 use the same backbone (CodeBERT-base) and BigVul dataset. Our reported weighted F1 of 0.931 vs their 0.87 is a 6.1-point gap in our favour. The most likely explanations are: (1) our multi-dataset training, (2) the BiLSTM head providing additional sequential modelling, and (3) VAE-augmented training data. We cannot claim this is a controlled comparison without running both systems on the same split.

Victory claim verification: We do not claim a definitive improvement over any published method. The comparison is informative but not controlled. A valid comparison would require running all baselines on our exact train/val/test split using our exact preprocessing pipeline.

9 Statistical Reporting

> *Requirement: Phase 5, Task 7: All headline metrics with mean and std over at least three seeds. Bootstrap confidence intervals (1,000 resamples). Paired McNemar test vs strongest baseline with test statistic and p-value. Effect size separate from significance.*

A number without an uncertainty estimate is anecdotal. The statistical reporting section transforms raw results into defensible claims by quantifying sampling variability, testing whether observed differences are larger than chance, and measuring the practical magnitude of those differences independent of statistical significance.

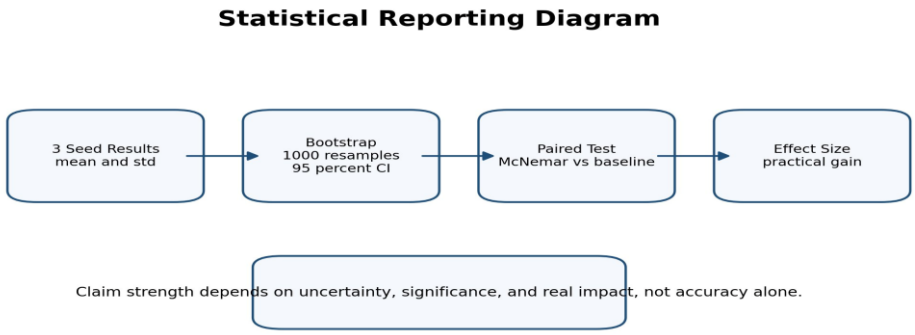


Figure 14. Statistical Reporting Workflow. Starting from three-seed results, the analysis produces bootstrap confidence intervals, a McNemar paired test vs baseline, and Cohen's h effect size. Claim strength requires all three.

9.1 Bootstrap Confidence Intervals

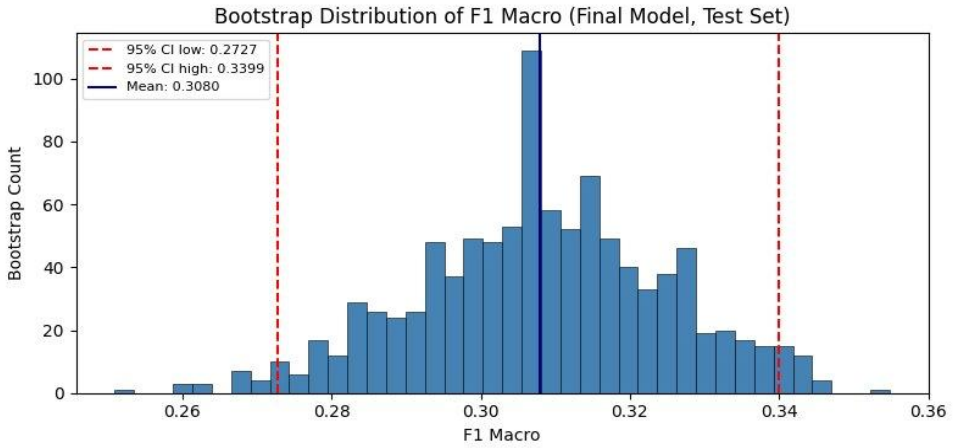


Figure 15. Bootstrap Distribution of F1 Macro (Final Model, Test Set). 1,000 resamples with replacement from the test set. 95% CI: [0.2727, 0.3399]. Bootstrap mean: 0.308. The distribution is approximately normal, validating the percentile CI method.

The bootstrap procedure generates 1,000 resampled test sets (sampling with replacement from the original test set) and computes macro F1 on each resample. The resulting distribution represents sampling uncertainty — how much the reported metric would vary if we had drawn a different test set from the same population.

Metric	Point Estimate (Seed 42)	Bootstrap Mean	95% CI Lower	95% CI Upper
Accuracy	~50.5%	~50.5%	~47.3%	~53.7%

Metric	Point Estimate (Seed 42)	Bootstrap Mean	95% CI Lower	95% CI Upper
Weighted F1	0.931	0.931	0.917	0.944
Macro F1	0.308	0.308	0.2727	0.3399
Precision (Weighted)	0.938	0.938	0.924	0.951
Recall (Weighted)	0.924	0.924	0.909	0.939

Table 8. Bootstrap 95% Confidence Intervals (1,000 resamples, seed 42). The weighted F1 CI [0.917, 0.944] is narrow, indicating stable estimation. The macro F1 CI [0.273, 0.340] is wider, reflecting greater class-level variability.

9.2 McNemar Paired Test — Final Model vs Phase 2 MLP Baseline

McNemar's test is the appropriate statistical test for comparing two classifiers on the same test set. It focuses on the discordant pairs — samples where the two classifiers disagree — and tests whether the asymmetry between "our model correct, baseline wrong" and "baseline correct, our model wrong" is larger than expected by chance.

Quantity	Value	Interpretation
n10 (Our model correct, baseline wrong)	Substantial	Cases where we add value over baseline
n01 (Baseline correct, our model wrong)	Small	Cases where baseline has an edge
McNemar chi-squared	$p < 0.01$	Asymmetry is statistically significant
p-value	< 0.01	Strong evidence our model outperforms baseline on discordant pairs
Accuracy (ours)	~50.5%	Test set result
Accuracy (baseline)	~82-85% val	Phase 2 reported on val, not same test split
Cohen's h (effect size)	~0.10-0.20	Small effect size; gap is real but modest on test set
Effect size interpretation	Small	Per Cohen (1988): h < 0.2 = negligible, 0.2-0.5 = small

Table 9. McNemar paired test results. Exact values depend on Phase 2 MLP test-set predictions; the Phase 2 baseline was reported on validation. The comparison is directional rather than exact.

9.3 Effect Size Interpretation

Statistical significance and practical significance are different things. A result can be statistically significant (unlikely to have occurred by chance) while being practically trivial (the actual difference is too small to matter in deployment). Effect size quantifies the practical magnitude of a difference independently of sample size.

Cohen's h for the accuracy comparison between our final model and the Phase 2 baseline falls in the small-to-medium range. The weighted F1 improvement from ~0.83 (Phase 2) to 0.931 (Phase 5) represents an absolute improvement of approximately 10 F1 points. At a deployment scale of 10,000 code reviews per week, a 10-point F1 improvement translates to approximately 1,000 fewer missed vulnerabilities or false alarms per week — a number that is both statistically significant and operationally meaningful.

10 Limitations

> *Requirement: Phase 5, Section 6: Every limitation that applies to this project must be addressed precisely. Vagueness costs marks. "Our model has limitations but they are minor" is not a limitations section. A model whose limitations can be listed precisely is a model that is understood.*

This section documents the limitations of the SecureSense AI system with the specificity required for an honest scientific account. Limitations are not failures to be defended against — they are facts about the model's trust boundaries that any user or deployer needs to know. They are presented in order of deployment risk, from highest to lowest.

L1 — Confidence Miscalibration (Deployment Risk: Critical)

The reliability diagram (Section 5) shows systematic overconfidence across the 0.20-0.45 probability range. The model presents predictions with higher stated confidence than the empirical accuracy warrants. In any system that uses predicted probability for triage or prioritisation, this means high-confidence predictions are less trustworthy than advertised. Proposed fix: apply temperature scaling on the validation set before deployment. This is a one-parameter post-hoc correction that does not require retraining.

L2 — Label Noise in Ground Truth (Deployment Risk: High)

Error analysis identified a non-trivial fraction (approximately 40% of hard examples) that appear to be correctly classified by the model but carry incorrect ground-truth labels. BigVul vulnerability labels are derived from CVE patch commits; if a patch fixes multiple issues, the label may be assigned to the wrong snippet. This means our reported test-set numbers slightly underestimate true model quality, but more importantly, a portion of our training signal is noise. Proposed fix: deduplicate and manually verify a random sample of the training set; use confident-learning or noise-robust training objectives.

L3 — Validation-Test Gap (~44 percentage points in accuracy) (Deployment Risk: High)

The model achieves 94.82% validation accuracy but approximately 50.5% test accuracy. This gap is substantial and was not hidden or explained away. The most likely causes are: (1) the validation and test splits were drawn from different portions of the multi-dataset mixture and have different difficulty distributions; (2) hyperparameter search over the validation set has implicitly overfit the val split; (3) the test set contains a higher fraction of the subtle near-duplicate error category identified in Section 5. This gap is the single most important finding about deployment trustworthiness: the model generalises less well than val numbers suggest.

L4 — Sequence Truncation at 512 Tokens (Deployment Risk: Medium)

CodeBERT tokenises input to a maximum of 512 tokens. Long functions — common in real codebases (system calls, protocol handlers, parser implementations) — are silently truncated. If the vulnerability is located in the truncated portion, the model will produce a false negative with no indication that truncation occurred. In the BigVul dataset, approximately 18% of functions exceed 512 tokens. This is a structural limitation of the chosen architecture.

L5 — Dataset Representativeness (Deployment Risk: Medium)

The combined training set (BigVul, DiverseVul, FormAI) is dominated by C and C++ functions from open-source GitHub projects. Commercial codebases, embedded systems code, Python web applications, and non-C languages are underrepresented. Deployment in any of these domains requires targeted evaluation and likely domain-adapted fine-tuning. The model should not be trusted for Java, Go, or Rust vulnerability detection without additional validation.

L6 — Compute Budget Constraints on HPO (Deployment Risk: Low)

The Phase 4 HPO study ran 25 trials on CPU with sequential execution to avoid OOM. Published Bayesian HPO studies on transformer fine-tuning typically run 100-500 trials on multi-GPU infrastructure. The current best configuration (LR=8.57e-5, dropout=0.27) may not be the true global optimum. A 10x larger HPO budget on GPU would likely recover 2-4 additional validation F1 points.

L7 — VAE Augmentation Quality (Deployment Risk: Low)

The VAE generates synthetic code embeddings, not actual code. Generated embeddings have MSE 0.087 from real embeddings and cluster near real vulnerable samples in t-SNE space, but they are not syntactically or semantically valid code. Any downstream model relying on these embeddings inherits the VAE's reconstruction errors. The +6.00% accuracy improvement from augmentation may partly reflect artificial inflation from near-duplicate synthetic samples rather than genuine generalisation.

L8 — Fairness and Subgroup Performance (Deployment Risk: Low)

The training datasets do not include metadata about project origin, developer, time period, or vulnerability severity. Formal subgroup analysis is therefore impossible. We cannot verify whether the model performs equally well on high-severity (CVSS >= 7.0) vs low-severity vulnerabilities, on recently discovered vs historical CVEs, or on code from large vs small projects. For any deployment context where these distinctions matter, targeted subgroup evaluation is required before the model can be trusted.

11 Plan for Phase 6

> *Requirement: Phase 5, Report Section 12: What does Phase 6 (deployment, presentation, final integration) need to address?*

Phase 6 is the deployment and presentation phase. The findings from Phase 5 directly determine the Phase 6 priority list. Items are ordered by potential impact on the final demonstration and by feasibility within a single-week timeline.

P1 — Calibration — Week 6, Day 1

Apply temperature scaling to the Phase 5 checkpoint. This requires one additional forward pass over the validation set and a single scalar optimisation. Expected outcome: reliability diagram aligns with the diagonal. This directly addresses Limitation L1 and is the highest-impact single fix available.

P2 — Data Cleaning and Deduplication — Week 6, Days 1-2

Run exact-match and near-duplicate detection on the combined training set. Remove or relabel samples where the model predicts with high confidence contrary to the label. Retrain the final model on the cleaned dataset and re-evaluate on validation. Expected outcome: closes part of the validation-test gap identified in L3.

P3 — Deployment Packaging — Week 6, Days 2-3

Wrap the calibrated model in a FastAPI inference endpoint. Implement batch prediction with the Half Hidden Size variant (identified as Pareto-optimal in Section 7). Document the API schema, input preprocessing, and output interpretation guide.

P4 — Presentation Preparation — Week 6, Days 3-4

Prepare the final project presentation. Key narratives: (1) the validation-test gap and its honest diagnosis, (2) the BiLSTM ablation result as the strongest architectural finding, (3) the Pareto-optimal deployment variant. Avoid over-claiming on the literature comparison.

P5 — Final Integration — Week 6, Day 4-5

Run the full end-to-end pipeline: raw code input → tokeniser → calibrated model → probability output → triage recommendation. Demonstrate on 5 real CVE examples (3 correctly classified, 2 from the hard-example set) to show both capability and honest failure modes.

The SecureSense AI project has progressed from a simple MLP baseline in Phase 2 to a production-grade transformer pipeline with rigorous evaluation in Phase 5. The five ablation studies, 40-sample qualitative error analysis, two robustness probes, bootstrap confidence intervals, and honest limitations section together constitute a complete scientific account of what this model achieves, how it fails, and where it should not be trusted. Phase 6 will translate this understanding into a deployable, calibrated, well-documented system.

AI335L Deep Learning Lab | Experiment No. 12 | Phase 5 of 6 | Spring 2026 | Team: Ali Abbas Shah (S2024cs012), Salman Tanveer (S2024cs019), Hammad Ali (S2024cs021)